Classifying Potential Exoplanets

Eddie Poma
Department of Computer Science
SUNY New Paltz
New Paltz, Ulster
Pomajre1@newpaltz.edu

ABSTRACT

Exoplanets are planets that reside outside of our own solar system, many even being part of their own solar systems. Over time, we have developed numerous methods to detect these exoplanets. However, due to the sheer distance that these exoplanets are away from us, these detection methods can often lead to false positives. What this essentially means is that we can identify objects of interest as either a confirmed exoplanet or false-positive based on their characteristics, making this a binary classification task where with enough data, we can try to predict whether an object of interest can be classified as an exoplanet or not. In this work, we utilize supervised machine learning models such as Naïve Bayes, logistic regression, SVM, and K-nearest neighbors (KNN) to predict the classification of numerous objects of interest based on their parameters. The dataset includes 9564 objects of interest as well as 37 useable features based on their astrophysical properties. Through a variety of methods, we were able to test how well each model performed with the task given to it. Every model performed above average with Naïve Bayes performing the worst with an accuracy of .60 and AUC of .785, followed by logistic regression and KNN both with an accuracy of .78 and AUC of .857 and .866 respectively. SVM performed the best with an accuracy of .81 and AUC of .888. The Naïve Bayes assumption proved to be a detriment when used with complicated real world data and performed notably worse. The other three models performed very similarly to each other and proved to be a good fit for this task.

I. INTRODUCTION

Exoplanets are planets that reside outside of our own solar system, many even being part of their own solar systems. Over time, we have developed numerous ways to detect these exoplanets with spectroscopy and photometry. However, due to the sheer distance that these exoplanets are away from us, these detection methods can often lead to false positives. Exoplanets could potentially be key search points for extraterrestrial life or habitable planets similar to Earth that we can colonize. By training a model to recognize what is or is not considered an exoplanet based on a number of astrophysical features, we can facilitate the process of finding more genuine exoplanets and focus research to find potentially hospitable ones.

In this work, we use several supervised machine learning models to predict whether or not an object of interest can be classified as an exoplanet or false positive based on a large number of astrophysical properties. Essentially, given an object of interest's features, how well do machine learning models perform when attempting to predict its classification? Since we are predicting between two possible outcomes of whether or not an object is an exoplanet, this task can be formulated as a binary classification problem. Given the complexity of the astrophysical features and the sheer number of objects in the dataset, machine learning models that perform well for this task would be ideal for further optimization and use for future projects.

For a binary classification task like this, we use machine learning algorithms such as Naïve Bayes, logistic regression, support vector machines (SVM), and k-nearest neighbors (KNN). We select a dataset publicly released by NASA of the Kepler space telescope's exoplanet findings as a csv file. This dataset contains detailed information for 9564 objects of interest that the telescope found over the course of its mission.

For this task, we divide the dataset into training and testing data in a 80/20 split. Additionally, we split the training data in another 80/20 split for training and validation data. Using this split data, we can train, compare, and validate how effective a specific machine learning model is. We can additionally analyze the results for further optimization to determine which model can perform the best for this task..

II. RELATED WORK

I was able to find an article that appeared to have a very similar motivation to what I wanted to achieve. This article called Logistic Regression to Boost Exoplanet Detection Performances [9] goes over the astrophysical process of detecting exoplanets. There is a lot of detail over the physics of the process, but they bring up the idea of extending a statistical framework with a logistic regression to filter out the list of candidates for exoplanets. They also mention image processing and clustering algorithms to get their results. This may be an idea worth pursuing in my own work. According to the article, using logistic regression, they were able to detect 50% more exoplanets with a specific feature they were looking for, with little noise from other candidates. This article is a good reference point for the idea of exoplanet detection using machine learning as well as advocating for using logistic regression for a task like this.

Additionally, there is another article with a similar presence called Exoplanet Detection Using Machine Learning Models Trained on Synthetic Light Curves [10]. This article goes over the benefits machine learning in the context of exoplanet detection. The algorithms of note are logistic regression, K-nearest neighbors, and random forest. Similarly to the other paper, this one also advocates the use of logistic regression for this task as well as getting good results using k-nearest neighbors and random forest. This article also uses the same or a similar Kepler space telescope dataset that I'm using as well. They go through a similar process that I did, making it a great resource to follow. These two articles convinced me that logistic regression and k-nearest-neighbors are a perfect fit for this type of task and should perform well.

III. Methods

This is a binary classification, where I want to train the model to recognize whether or not an object of interest can be classified as an exoplanet or not based on the given input data. I will be choosing to train this data under the Naïve Bayes, logistic regression, SVM, and K-nearest neighbors models.

To begin, I'm trying to do a binary classification so we need to examine the dataset to see which parameter determines the classification for each object. This would be 'koi_disposition', which contains three classifications of 'confirmed', 'candidate', and 'false positive'. To fit this into a binary classification, we consolidate 'confirmed' and 'candidate' to be 'yes', and 'false positive' to be 'no'. We can create a new column called 'target' that is filled with zeroes and ones, representing 'false positive' and 'confirmed/candidate' respectively. After deleting any row that didn't contain a one or zero, we are left with 5023 zeroes and 4521 ones.

Next we need to filter out and delete any features that do not contribute to the training. In this case, there are numerous columns dedicated to things such as names or IDs of the objects. These can be safely excluded. However there is also the problem of features that are too strong to the point where the model will always have a perfect result with no error, which is not realistic for what we want to achieve. Features such as the previously mentioned 'koi_disposition' give away the classification of each object, so we must exclude it from our usable features. After filtering out irrelevant and strong features, we are left with 37 columns.

The dataset chosen is heterogenous, meaning it contains various types of data, in this case, the features are both numeric and categorical. We separate these two types of data, making sure to isolate the 'target' column, as having it in the training data would essentially be giving the model the answers, which we want to avoid. We fill in the empty numeric columns with the medians of their respective columns to avoid large outliers affecting the data too much. Likewise, we fill empty categoric columns with 'MISSING'. This is to prevent the model from potentially running into NaN or Null values. After finalizing the cleaning of the data, we randomize the order of the columns to avoid circumstances where the dataset was arranged in a specific order, which would affect the model's ability to learn. Through the use of categorical preprocessing and pipelines, we are able to convert the categorical data into binary values, making them compatible with the various models, as string/text are not.

Our features of this dataset are the combination of the numeric and categorical columns, with our outcome variable being the 'target' column. From this, we need to get our test data, training data, and validation data. We do this through train test splitting, where 20% of the data is the test data to be set aside for later use. 20% of the remaining 80% will be the validation data, and the remaining data will be the training data.

Now we can begin the model's training using Naïve Bayes. Naïve Bayes functions under the assumption that every feature is independent of each other. The formula for Naïve Bayes is as follows:

$$P(y|x_1, x_2, \dots, x_n) = \frac{P(y)\prod_{i=1}^{n} P(x_i|y)}{P(x_1,x_2,\dots,xn)}$$

Due to the assumption that Naïve Bayes has, it makes this model very quick and efficient for binary classification tasks with a large amount of data and features such as this. The naïve assumption does become detrimental for complicated real world data such as the dataset we're using, so this model may not perform the best despite its efficiency.

We can also train this data using the logistic regression model. Logistic Regression is a model that is especially suited for binary classification tasks such as this, as it can only produce one of two results, in this task's case that being yes/no for an exoplanet prediction. Logistic regression makes use of the sigmoid function to convert inputs into a probability between 0 and 1. The sigmoid function is as follows:

$$\textit{Where } g(z) = \frac{1}{1+e^{-z}}$$

Support vector machines (SVM) models are also a good fit for binary classification tasks. This model attempts to find the best boundary or hyperplane that separates different classes in the data. Since SVMs essentially split two classes, it is a perfect fit for this task. In addition, SVMs are effective for sporadic data with a lot of outliers since the only data it cares about are hyperplane split. SVM does struggle with noisy datasets with overlapping classes, so a thoroughly cleaned dataset and caution must be used before training with it.

One important note when using SVM is that there a number of different kernels you can use to optimize results depending on the data being trained. Since this dataset contains all sorts of nonlinear physical relationship features, the radial basis function (RBF) kernel would fit best since it handles nonlinear boundaries well. RBF has two critical parameters, C and gamma. C (regularization) trades off misclassification of training data against maximization of the decision function's margin. A low C makes the decision surface smooth, and a high C aims at classifying all training data correctly. In other words, a large C leads to potential overfitting, and a small C gives more generalization. Gamma defines how far the influence of a single training sample reaches and influences the shape of the decision boundary. For sake of simplicity, we are using the default parameters C=1 and gamma='scale' for RBF. Keep in mind that these parameters can very much be optimized for the best results by fine tuning them. However, by utilizing a GridSearch, we can attempt to find an optimal C and gamma value for the SVM model. In the GridSearch we attempt to find the parameters that would give the best AUC score based on our dataset and training data.

K nearest-neighbors is an interesting model that can work well for binary classification tasks. The way it works is that the model searches for the nearest "k" number of data points or "neighbors" to a given input and makes predictions based on the majority class or average value. The "k' is simply a number that tells the algorithm how many neighbors to look for before making a decision. For example, for a new datapoint there are 2 class A and 1 class B neighbors nearby. If k=3, then the algorithm will classify this new datapoint as class A since the

closest 3 neighbors are class A. The value of k is a critical factor for this algorithm to get good results. A large k can help with stabilizing the predictions for datasets that contain a large amount of noise or outliers, but too large of a k can lead to overfitting. For a binary classification specifically, it is important for the value of k to be an odd number to prevent ties from happening. In addition to this, another important part of KNN is the distance metric used to find the nearest neighbor. This distance must be defined before a classification can be made. This is important since these distance metrics help form decision boundaries. There are several distance metrics to choose from but the two most common are the Euclidean distance and Manhattan distance, both of which can be represented in a generalized form called the Minkowski distance. The formula for the Minkowski distance is as follows:

$$\text{Minkowski Distance} = \left(\sum_{i=1}^{n} |x_i - y_i|\right)^{1/p}$$

The parameter, p, represents what distance metric will be used, where p=1 gives the Manhattan distance and p=2 gives the Euclidean distance. The Euclidean is the most common distance metric which simply measures a straight line between the query point and the other point being measured. The Manhattan distance measures the absolute distance value between two points. This is often preferred for high dimensional data and is less sensitive to noise and outliers.

In addition to testing the model's general performance, I also wanted to experiment with how much impact the value of k and the chosen distance metric affect's KNN's performance in terms of its confusion matrix metrics and ROC-AUC score. To do this, I tested the model using the Manhattan and Euclidean distances with each distance using 4 different k values of k=5, k=25, k=51, and k=101 for a total of 8 models to analyze.

To interpret the results and performance of each of the models, I chose to make use of confusion matrices and AUC-ROC scores. The confusion matrix can be used to measure how well a classification model is performing by comparing the predictions made with the actual results. Predictions are broken down into four categories: true positive (TP), true negative (TN), false positive (FP), and false negative (FN). Using these four categories, we can further calculate useful metrics such as accuracy, precision, recall, and F1-score to further analyze how well the model performs. Receiver Operating Characteristic (ROC) curves are graphs used to check how well a classifier model performs. It plots true positive rate (TPR) against false positive rate (FPR). These two are calculated using the previously mentioned four metrics from the confusion matrix, that being TP, FP, TN, and FN. The equations to calculate FPR and TPR are as follows:

$$FPR = \frac{FP}{FP + TN} \quad TPR = \frac{TP}{TP + FN}$$

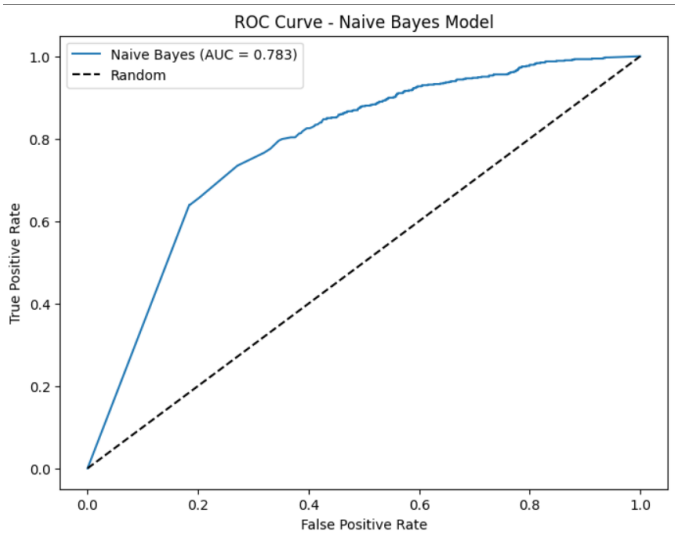
The higher an ROC curve is above the random classifier (0.5) line, the better the results. Additionally using the area under the curve (AUC) of the ROC we can determine how well a model is performing. A higher AUC typically indicates better model performance, with an AUC of 1.0 being perfect, while 0.5 is random guessing.

IV. Results

Upon training the model under Naïve Bayes, I made use of the classification report in scikit to produce a confusion matrix to see its metrics. The confusion matrix stated that this model had a precision percentage of 87% for negative classifications, but 57% for positive classifications. Additionally, the recall was 27% for negatives and 95% for positives. The confusion matrix produced can be seen below:

```
Classification report:
              precision    recall  f1-score   support

           0       0.87      0.27      0.41       804
           1       0.54      0.95      0.69       727

    accuracy                           0.60      1531
   macro avg       0.70      0.61      0.55      1531
weighted avg       0.71      0.60      0.55      1531

Confusion matrix:
 [[219 585]
 [ 34 693]]
```

The results for the Naïve Bayes model were efficient but inconsistent across the board in terms of the confusion matrix metrics. The accuracy of 60% is only slightly better than a coinflip. In addition to the confusion matrix, an ROC-AUC graph and score was produced using this confusion matrix which can be seen below:

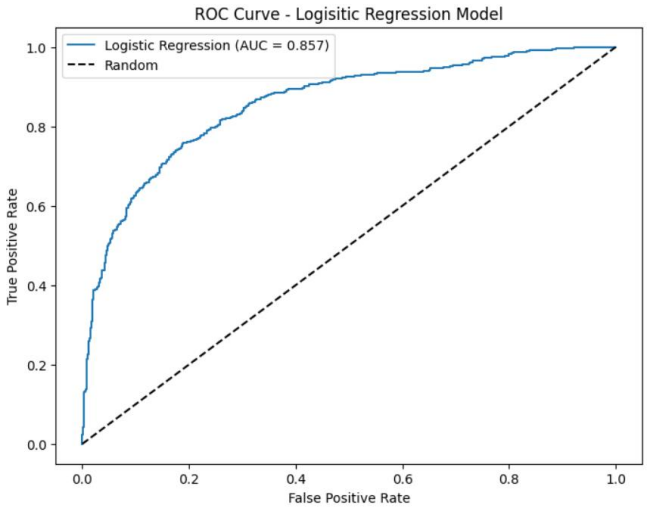


We can see from the shape of the graph and its ROC-AUC score of 0.783 that the model generally got most of its predictions correct. A cause for this model's mediocre performance is the naïve assumption being a detriment for complicated real world data such as the astrophysical features for this problem's dataset. These results should form a baseline for how bad the results of the other models should not be. Seeing as the other models do not deal with the naïve assumption, the Naïve Bayes model should be the worst performing one by a decent margin.

When training the model with Logistic regression, I repeated the same calculations to get the results as I did for Naïve Bayes. A confusion matrix for the logistic regression model can be seen below:

```
=== Logistic Regression Evaluation ===
              precision    recall  f1-score   support

           0       0.80      0.76      0.78       804
           1       0.75      0.79      0.77       727

    accuracy                           0.78      1531
   macro avg       0.78      0.78      0.78      1531
weighted avg       0.78      0.78      0.78      1531

Confusion matrix:
 [[611 193]
 [150 577]]
```

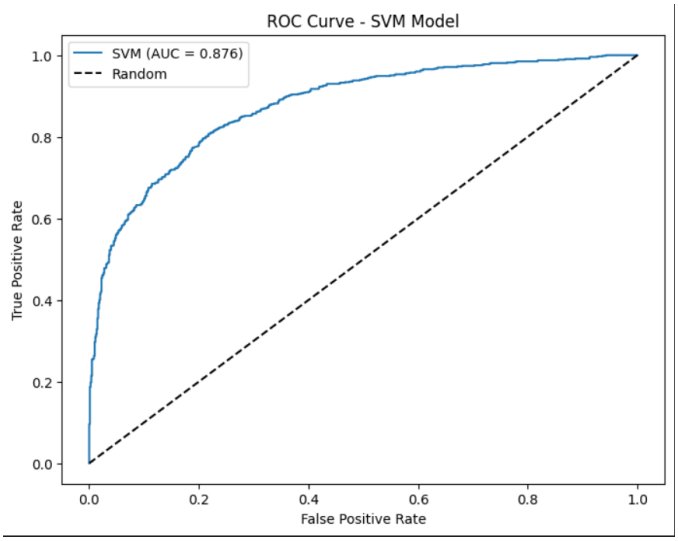
When examining the confusion matrix, the results for precision and recall were much more consistent with each other, with the highest percentage being 80% precision for negative classifications. The other results were within 5%. The accuracy of 78% is significantly better than Naïve Bayes' 60%. These confusion matrix metrics are consistent with each other across the board. An ROC-AUC curve and score were produced using these metrics which can be seen below:



Looking at the shape of the curve and its AUC score of 0.857, this model performed much better than Naïve Bayes did, which is a good sign given that the naïve assumption is not a factor for this model as well as logistic regression being a perfect fit for binary classification tasks such as this. These results are a good baseline for determining if the other models can perform well for this task.

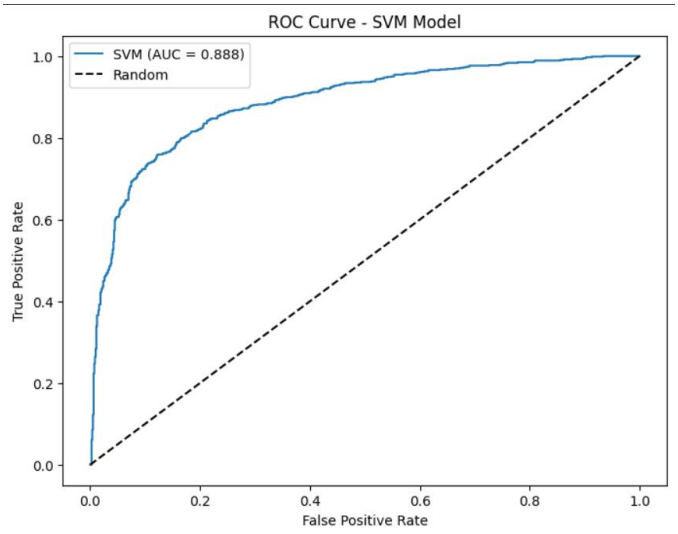
SVM was tested with two different models using different parameters for C and gamma. The first model was tested using the default values C=1, gamma='scale'. The second model was tested using the C and gamma values found by the GridSearch which were found to be C=50 and gamma=0.01. The default value's confusion matrix and its ROC-AUC curve can be seen below:

```
=== SVM Evaluation ===
              precision    recall  f1-score   support

           0       0.80      0.80      0.80       804
           1       0.78      0.78      0.78       727

    accuracy                           0.79      1531
   macro avg       0.79      0.79      0.79      1531
weighted avg       0.79      0.79      0.79      1531

Confusion matrix:
 [[643 161]
 [160 567]]
```



The default SVM model's confusion matrix metrics are consistent across the board with all metrics being within 0.02 of each other, as well as having an accuracy of 79% which slightly outperforms logistic regression.. The ROC-AUC score of 0.876 also outperforms logistic regression. The default values of the RBF kernel are typically not the greatest, but even with them this SVM model outperformed logistic regression. By optimizing the values for C and gamma, we can get even better results. The confusion matrix for the optimized model and its ROC-AUC curve can be seen below:

```
=== SVM Evaluation ===
              precision    recall  f1-score   support

           0       0.83      0.82      0.82       804
           1       0.80      0.81      0.81       727

    accuracy                           0.81      1531
   macro avg       0.81      0.81      0.81      1531
weighted avg       0.81      0.81      0.81      1531

Confusion matrix:
 [[657 147]
 [137 590]]
```



Comparing the default values with the optimized ones, the optimized model does outperform the default model in both the confusion matrix metrics and its ROC-AUC score. For the confusion matrix metrics, we can see that the optimized model doesn't have a single metric below 0.80, whereas the default model does have a couple of metrics at 0.78. The accuracy of the optimized model of 81% is slightly better than the default model's 79%. Finally, the AUC of the optimized model is 0.888, which is 0.012 higher than the default model's 0.876. Overall, optimizing the model for a better C and gamma did prove to be effective in producing better performance than the default values. Regardless of the values chosen for C and gamma, the SVM model outperformed logistic regression.

As stated previously, KNN was tested using 8 models using the two distance metrics and 4 different k values respectively. For the Euclidean distance, the following confusion matrices were produced:

```
=== K=5 Nearest Neighbors Evaluation ===
              precision    recall  f1-score   support

           0       0.80      0.78      0.79       804
           1       0.77      0.78      0.77       727

    accuracy                           0.78      1531
   macro avg       0.78      0.78      0.78      1531
weighted avg       0.78      0.78      0.78      1531

Confusion matrix:
 [[630 174]
 [159 568]]
```

```
=== K=25 Nearest Neighbors Evaluation ===
              precision    recall  f1-score   support

           0       0.80      0.78      0.79       804
           1       0.76      0.79      0.77       727

    accuracy                           0.78      1531
   macro avg       0.78      0.78      0.78      1531
weighted avg       0.78      0.78      0.78      1531

Confusion matrix:
 [[628 176]
 [156 571]]
```

```
=== K=51 Nearest Neighbors Evaluation ===
              precision    recall  f1-score   support

           0       0.80      0.77      0.78       804
           1       0.75      0.79      0.77       727

    accuracy                           0.78      1531
   macro avg       0.78      0.78      0.78      1531
weighted avg       0.78      0.78      0.78      1531

Confusion matrix:
 [[616 188]
 [155 572]]
```

```
=== K=101 Nearest Neighbors Evaluation ===
              precision    recall  f1-score   support

           0       0.81      0.75      0.78       804
           1       0.74      0.80      0.77       727

    accuracy                           0.77      1531
   macro avg       0.77      0.77      0.77      1531
weighted avg       0.78      0.77      0.77      1531

Confusion matrix:
 [[603 201]
 [146 581]]
```

ROC curves were created for each of these matrices and their ROC-AUC scores were calculated to be the following: k=5 AUC=0.836, k=25 AUC=0.852, k=51 AUC=0.846, k=101 AUC=0.834.

For the Manhattan distance, the following confusion matrices were produced:

```
=== K=5 Nearest Neighbors Evaluation ===
              precision    recall  f1-score   support

           0       0.80      0.78      0.79       804
           1       0.77      0.78      0.77       727

    accuracy                           0.78      1531
   macro avg       0.78      0.78      0.78      1531
weighted avg       0.78      0.78      0.78      1531

Confusion matrix:
 [[630 174]
 [159 568]]
```

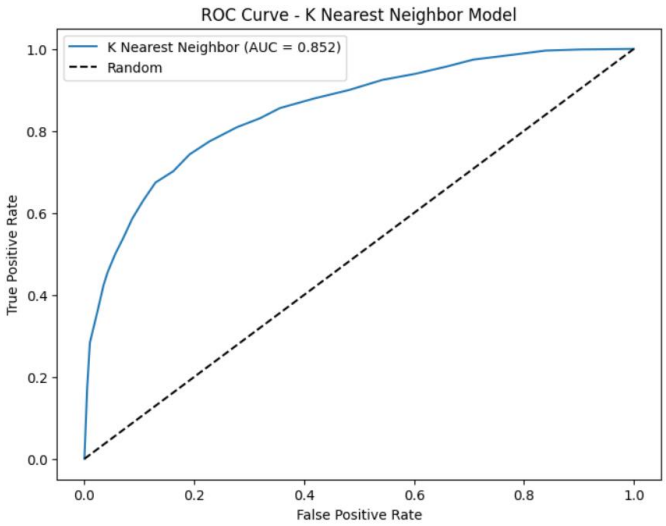
```
=== K=25 Nearest Neighbors Evaluation ===
              precision    recall  f1-score   support

           0       0.80      0.78      0.79       804
           1       0.76      0.79      0.77       727

    accuracy                           0.78      1531
   macro avg       0.78      0.78      0.78      1531
weighted avg       0.78      0.78      0.78      1531

Confusion matrix:
 [[628 176]
 [156 571]]
```

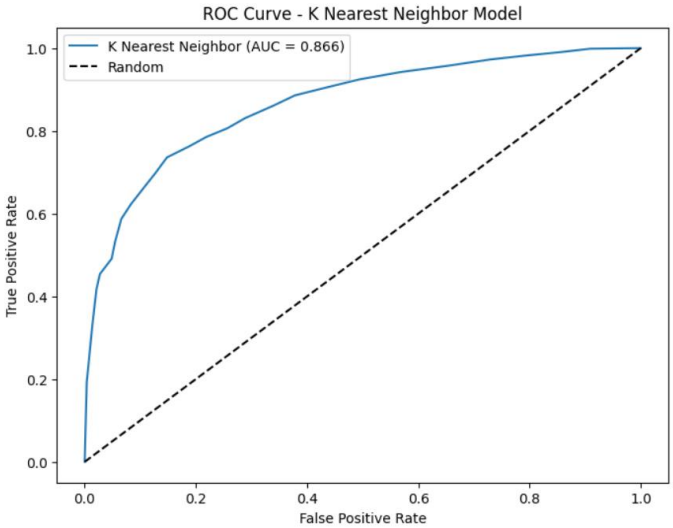
```
=== K=51 Nearest Neighbors Evaluation ===
              precision    recall  f1-score   support

           0       0.80      0.77      0.78       804
           1       0.75      0.79      0.77       727

    accuracy                           0.78      1531
   macro avg       0.78      0.78      0.78      1531
weighted avg       0.78      0.78      0.78      1531

Confusion matrix:
 [[616 188]
 [155 572]]
```

```
=== K=101 Nearest Neighbors Evaluation ===
              precision    recall  f1-score   support

           0       0.81      0.75      0.78       804
           1       0.74      0.80      0.77       727

    accuracy                           0.77      1531
   macro avg       0.77      0.77      0.77      1531
weighted avg       0.78      0.77      0.77      1531

Confusion matrix:
 [[603 201]
 [146 581]]
```

ROC curves were created for each of these matrices and their ROC-AUC scores were calculated to be the following: k=5 AUC=0.850, k=25 AUC=0.866, k=51 AUC=0.862, k=101 AUC=0.856.

Comparing both sets of models, we can see that the confusion matrix metrics are very consistent across the board regardless of the distance metric chosen or k-value chosen. In addition, the accuracies are consistent with each other as well, ranging from 75%-78%. The AUC scores are where we can see the finer details of how each individual model performed. Looking at the AUC scores, we can see that k=25 performed the best for both distance metrics, with k=5 and k=101 performing similarly to each other and worse than k=25 and k=51. Comparing the AUC scores for each distance metric, we can see that the AUC scores for the Manhattan distance outperformed the Euclidean distance regardless of the k chosen. The ROC-AUC curves for the best performing models of k=25 for each distance metric can be seen below:





The best performing KNN model out of all 8 models tested was using the Manhattan distance with k=25, producing an AUC of 0.866. In general the KNN model performed well and consistently regardless of distance metric and k value. The best model produced using the Manhattan distance and k=25

performed slightly worse than the optimized and default SVM models, and performed slightly better than the logistic regression model.

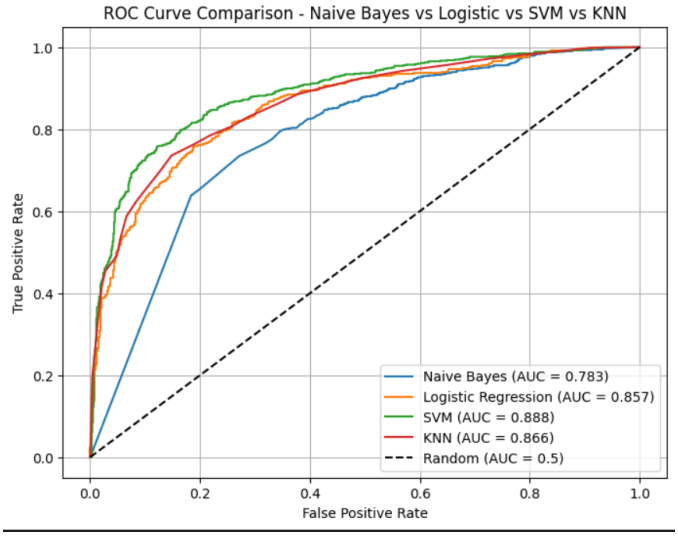
V. Discussion and Conclusion

These results are consistent with how the these models were presented. Naïve Bayes is quick and efficient, especially for this task. However, while the model was mostly correct in its predictions, other results such as its 60% accuracy and confusion matrix metrics were very lopsided towards either positive or negative classifications. This most likely has to do with the assumption of Naïve Bayes not being applicable in a real world setting such as astrophysics. This model's AUC score of 0.783 was surprisingly good and is roughly 0.070 behind the other models. Naïve Bayes performed decently overall, but was by far the worst performing model by a large margin and should not be used for the task with this dataset in the future.

Logistic regression was much more consistent and accurate with its predictions, which is to be expected since the task provided to it was a perfect fit for this type of model. Surprisingly, it performed slightly worse than both SVM and KNN even when both model's parameters are unoptimized. But it should be noted that all of logistic regression, SVM, and KNN performed very similarly to each other in terms of their confusion matrix metrics, accuracies, and AUC scores. While it did very well overall, logistic regression performed the worst out of these three models with an AUC of 0.857.

The KNN experiment with testing various values with the two different distance metrics proved that KNN's performance changes slightly depending on the value of k and the distance measure used, but regardless of which, all gave good performances that compete with logistic regression and SVM. The best model produced used the Manhattan distance with k=25. Seeing as how this dataset is large with a lot of noise and outliers, the Manhattan distance is a good choice of distance metric for a KNN model to perform this task. The optimal value of K seems to be between 25 and 51.

The SVM model performed the best out of all 4 models tested, even when it was not using an optimal C or gamma value. When optimized however, SVM performed the best by a good margin compared to the best model produced by KNN with an AUC of 0.888 compared to KNN's 0.866, which is a 0.022 difference. This is notably a much larger difference in AUC score than that between logistic regression and KNN which is only a 0.009 difference. An ROC-AUC curve overlapping all 4 tested models to compare can be seen below. Note that the SVM and KNN models represented are the best performing ones that were produced.



For the purpose of using this dataset to determine whether or not an object of interest is an exoplanet, the naïve assumption in Naïve Bayes is too detrimental for this task and should be avoided. Logistic regression, KNN, and SVM models are a good fit for this task and should be explored and optimized.

References

[1] Logistic Regression in Machine Learning - GeeksforGeeks

[2] Naive Bayes Classifiers - GeeksforGeeks

[3] Kepler Mission Data Resources in the Exoplanet Archive

[4] Column Transformer with Mixed Types — scikit-learn 1.7.2 documentation

[5] ColumnTransformer — scikit-learn 1.7.2 documentation

[6] Logistic Regression using Python - GeeksforGeeks

[7] Pros and Cons of Naive Bayes - EducationalWave

[8] What Is Logistic Regression? | IBM

[9] Hadrien Cambazard, Nicolas Catusse, Antoine Chomez, Anne-Marie Lagrange, Logistic regression to boost exoplanet detection performances, *Monthly Notices of the Royal Astronomical Society*, Volume 536, Issue 2, January 2025, Pages 1610–1624, https://doi.org/10.1093/mnras/stae2657

[10] Exoplanet Detection Using Machine Learning Models Trained on Synthetic Light Curves

[11]https://scikitlearn.org/stable/auto_examples/svm/plot_rbf_parameters.html

[12] https://www.geeksforgeeks.org/machine-learning/support-vector-machine-algorithm/

[13] https://scikit-learn.org/stable/modules/svm.html

[14]https://www.geeksforgeeks.org/machine-learning/performing-feature-selection-with-gridsearchcv-in-sklea